CS411 Software Architecture Design

2026–2027 Fall Semester, Project 1

Group member	Student ID	E-mail
Leyla Abbasova	22301172	leyla.abbasova@ug.bilkent.edu.tr
Zaeem Masood Sheikh	22201178	masood.sheikh@ug.bilkent.edu.tr
Farrukh Mammadov	22301186	farrukh.mammadov@ug.bilkent.edu.tr
Shahin Ibrahimli	22301574	shahin.ibrahimli@ug.bilkent.edu.tr

Contents

1. Why we chose Ollama..... 3
2. Current architecture..... 3
2.1 Problem definition and domain 3
2.2 Major technical problems and the project's solutions 3
2.3 Stakeholders 4
2.4 Quality requirements 5
2.5 Architectural views 6
2.6 Rationale behind the architecture 9
2.7 Architectural patterns 10
3. Extension: an observability layer.....10
3.1 Choice of evolution area and reasoning 10
3.2 Architectural change 11
3.3 Rationale behind the new architecture 13
3.4 Architectural patterns in the new architecture 13
4. Login page and IAM module..... 14
4.1 Requirements derived from Ollama 14
4.2 Architecture 15
4.3 The login page 18
4.4 Implementation 20
4.5 Verification and known limits 22
5. Conclusion..... 23
References.....24

1. Why we chose Ollama

Out of the five options, we picked Ollama because it is the only one that is really about a resource scheduling problem on a single machine. It has to fit language models that are several gigabytes in size into whatever memory the user has, and it hides all of this behind a single command: `ollama run llama3.2`. Terraform, Argo CD, Vault and Kubernetes are mature infrastructure tools whose architectures are already well documented. Ollama is newer, changes quickly and its architecture is still being worked out in public, so we thought it would be a more interesting system to study. We had four main reasons.

- Its architectural decisions are easy to see. In the version we studied, Ollama runs llama.cpp's own `llama-server` as a subprocess, and it recently added a second engine for Apple's MLX framework behind the same `LlamaServer` interface. This is a real example of an interface taking on a second implementation, which is the kind of evolution we talked about in the lectures.
- It is a manageable size. The Go code base has about 435 non-test source files with clearly separated packages, so as a student team we could actually read the important code paths from start to finish.
- It has real gaps that fit the assignment. Ollama has no user authentication and no built-in metrics endpoint, and the community has been asking for both for a long time. This means our extension in Part 3 and our IAM module in Part 4 solve problems the project actually has, not made-up ones.
- We could run it ourselves. Every one of us installed it on a laptop and checked the behaviour we describe in this report. This would have been much harder with the other options, especially Kubernetes and Argo CD.

2. Current architecture

2.1 Problem definition and domain

Ollama is a tool for local LLM inference. In other words, it runs large language models on hardware that the user controls, instead of sending requests to a hosted API. People use it because their prompts never leave their machine, there is no cost per token, it works offline and they are free to choose whichever models they like.

Running an open-weights model by hand takes a lot of work. You need to choose a file format and a quantisation level, compile an inference engine for the right GPU backend (Vulkan), estimate how many layers will fit in video memory, write the chat template for the model and then find a way to expose it to applications. Ollama handles all of these steps. It downloads models from a registry in the same way Docker pulls images, works out where to place them on the available hardware, keeps them loaded between requests and serves them through a stable HTTP API. The API is available in three forms: Ollama's native API, an OpenAI-compatible API and an Anthropic-compatible API. Users can also write a `Modelfile`, which is intentionally modelled on a Dockerfile, to create their own version of a model with a different system prompt, parameters or adapters.

2.2 Major technical problems and the project's solutions

The table below lists the main technical problems we found while reading the code and how Ollama deals with each of them.

Problem	How Ollama solves it
Models are larger than the memory of many machines	It uses quantised GGUF weights and memory-mapped loading. The scheduler predicts how much VRAM is needed, chooses a GPU or a group of GPUs, offloads the remaining layers to the CPU, and if a load runs out of memory it tries again with a smaller automatic context.
Hardware is very different from user to user	The `discover` package probes the devices on each platform, and inference runs in bundled backend libraries that are chosen at run time. On Apple silicon there is also the MLX engine.
Several models and requests compete for one GPU	A single scheduler goroutine makes every load and unload decision. Requests arrive on channels and are queued, they can share a loaded runner, and models that are idle are removed after a keep-alive period.
The inference engine can crash or leak memory	Each loaded model runs in its own subprocess, and the server talks to it over local HTTP. If one crashes, only that model goes down, not the whole server. The scheduler notices that the process exited and loads it again.
Models are several gigabytes and need to be distributed	Blobs are content-addressed and each tag has a JSON manifest, which is a layout similar to OCI images. Shared layers are only stored once, and downloads are split into chunks, run in parallel and can be resumed.
Every model family has its own prompt format, tool-call format and "thinking" syntax	Go templates are stored as a layer in the manifest, and there are model-specific renderers and output parsers.
Many applications are written for OpenAI's or Anthropic's API	Translation middleware converts `/v1/chat/completions`, `/v1/responses` and `/v1/messages` into native requests and converts the answers back.
Web pages in a browser can reach an API on localhost	By default the server binds to 127.0.0.1, checks allowed hosts on loopback and uses a CORS allow-list. Pushes to the registry are signed with an ed25519 key that is created per install.

2.3 Stakeholders

We identified the following stakeholders, what they expect from Ollama and what they give back to the project.

Stakeholder	Expectations	Contributions
Core maintainers	A simple product, a stable API, quick support for new model releases and a sustainable business.	The architecture, most of the commits, releases, code review, the registry and the desktop apps.
Community contributors	Their pull requests get reviewed, and features such as metrics, authentication and new backends get added.	Bug fixes, ports to new platforms, issues, documentation and translations of other APIs.
End users and developers	Installing with one command, good speed on their own hardware, privacy and no cost.	Bug reports, benchmarks, recommending it to others and sharing Modelfiles on the registry.
Application integrators	The API stays compatible across releases, with streaming, tool calling and structured output.	Most of Ollama's reach comes from them, along with client libraries and feedback on the API design.
Upstream projects	Correct attribution, and fixes sent back upstream.	The inference kernels that Ollama depends on.
Model publishers	Their models run correctly, with the right template, on the day they are released.	Open weights, chat templates and sometimes direct work together on day-one support.

Stakeholder	Expectations	Contributions
Hardware vendors	Their accelerators are detected and used well.	Drivers, SDKs and occasionally engineering help.
Operators in companies and labs	Running Ollama as a shared service with access control, monitoring, auditing and predictable resource use.	Deployment recipes and feature requests such as #3144.
Security researchers	Reports are handled responsibly.	Findings such as unauthenticated servers left open on the internet and memory bugs caused by malformed GGUF files.

2.4 Quality requirements

We ranked the quality attributes by how clearly the code shows the maintainers optimising for them. For each one we wrote a concrete scenario, following the quality attribute scenario technique from the lectures.

Attribute	Scenario	Our reasoning
Usability	A new user types `ollama run llama3.2` on a fresh laptop. The model downloads, loads and answers without any configuration.	This is the top priority. Every default exists so that users never have to tune anything.
Performance	A model that is already loaded receives a chat request. The first token arrives in well under a second, and generation speed is close to raw llama.cpp.	Keep-alive, reusing runners and a thin Go layer on top of native kernels keep the overhead small. Cold loads are the main cost, which is why models are evicted lazily.
Resource efficiency	A 14B model is requested on a 12 GB GPU. It loads with part of it offloaded to the CPU instead of failing.	Memory estimation, retrying with a smaller context after an out-of-memory error, and a quantised KV cache.
Portability	The same release runs on macOS, Windows, Linux and Docker, on CUDA, ROCm, Metal, Vulkan or CPU.	Devices are discovered at run time and there is a library per backend. Because the server is written in Go, cross-compiling it is easy.
Interoperability	An app written for the OpenAI SDK only changes its base URL, and it works.	Translating at the edge is cheaper than asking the whole ecosystem to adopt a new API.
Modifiability	A new model architecture is released, and support ships within days.	The engine sits behind the `LlamaServer` interface, templates and parsers are written per model family, and upstream llama.cpp is followed closely.
Availability / fault isolation	A runner crashes on a malformed input. Only that request fails and the server keeps serving.	There is one subprocess per model, and the scheduler detects when it exits and frees the slot.
Security	A web page in the user's browser tries to call the local API. It is blocked by the host and origin checks.	It is secure for a single local user. As soon as the port is exposed, though, there is no authentication at all, and that is what Part 4 is about.
Scalability	Many users share one server. Requests wait in the queue, and scaling out needs an external load balancer.	This is left out on purpose, since Ollama is designed to run on a single node.

Attribute	Scenario	Our reasoning
Observability	An operator wants to see request rate, queue depth and GPU memory over time. Only logs and `/api/ps` exist.	This is the weakest attribute today, and it is the motivation for Part 3.

Looking at the ranking, the main trade-off in the architecture becomes clear. Ollama spends most of its complexity on usability, performance and portability for a single user, and accepts weak security, scalability and observability when it is shared. Our extension and our IAM module both target the shared-use side, and since both of them are opt-in they do not make things worse for a single user.

2.5 Architectural views

We used Kruchten's 4+1 view model. The scenarios connect four other views: a logical view, a development view, a process view and a physical view. Ollama does not publish any official architecture diagrams, so we drew all of the diagrams ourselves in PlantUML after reading the source code.

Scenario view

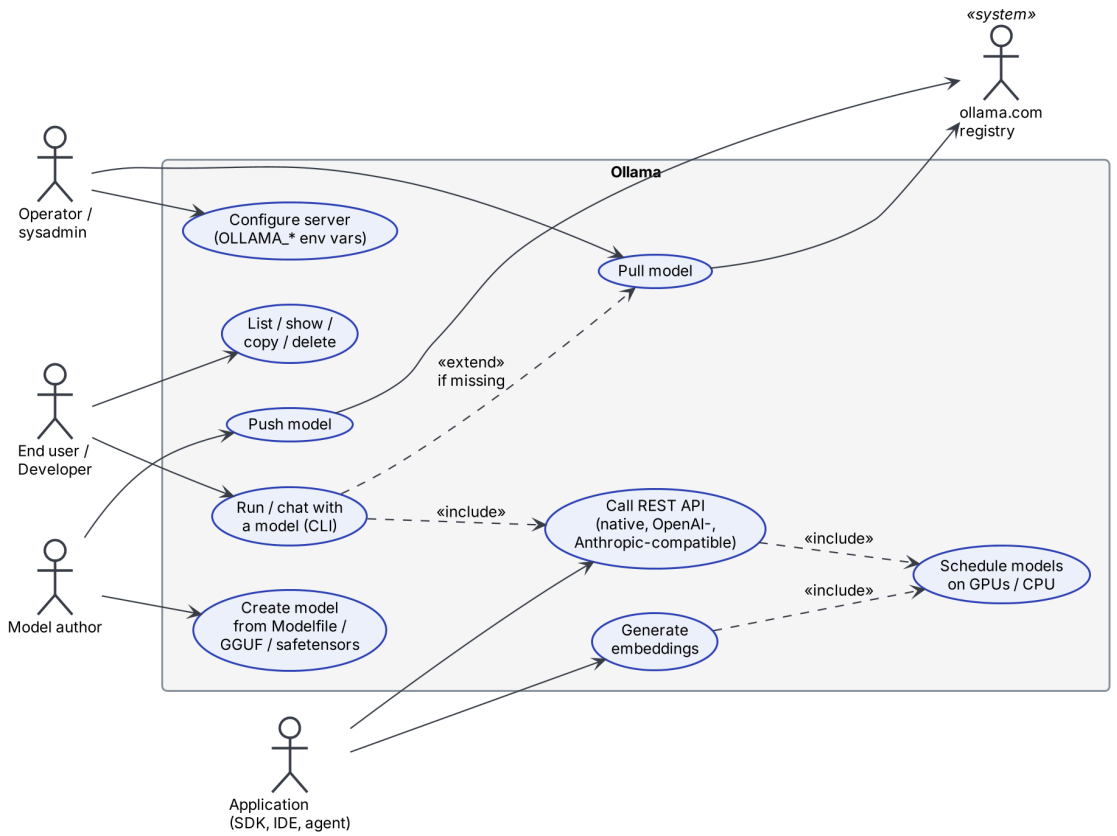


Figure 1. Use cases. Running a model includes calling the API and scheduling, and pulling extends it when the model is missing.

Development view

The repository is organised by responsibility rather than by layer, but four groups can still be seen. These are the clients, the HTTP layer, model handling and execution. Dependencies point downwards, and the only package that is read from everywhere is `envconfig`.

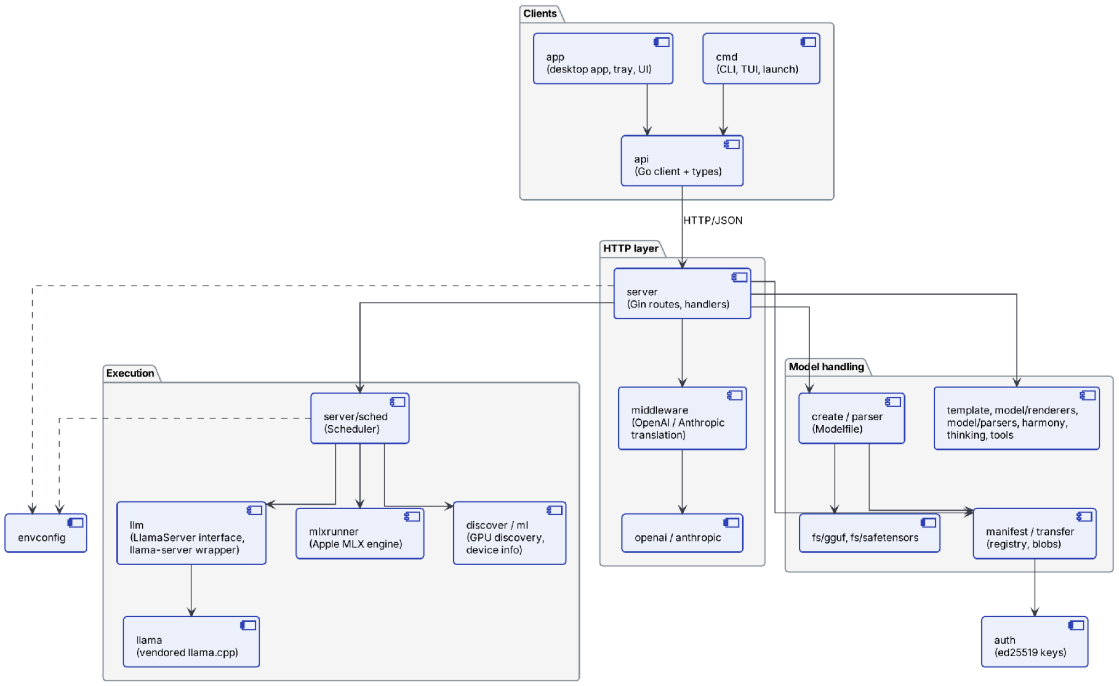


Figure 2. Package view of the Go code base.

Logical view

The core idea is a split between what to run and where it runs. The `Scheduler` is the only class that creates or destroys runners. Handlers only borrow them, and a reference count makes sure of this. `LlamaServer` is an interface with two implementations, so the engines can be swapped.

Physical view

Everything runs on the user's own machine. The server is a single Go process, and every loaded model is a child process that maps the weights from the local store and talks to the accelerator. The only network dependencies are the registry (for pull and push, signed with the install's ed25519 key) and the optional cloud models, which can be turned off with `OLLAMA_NO_CLOUD`.

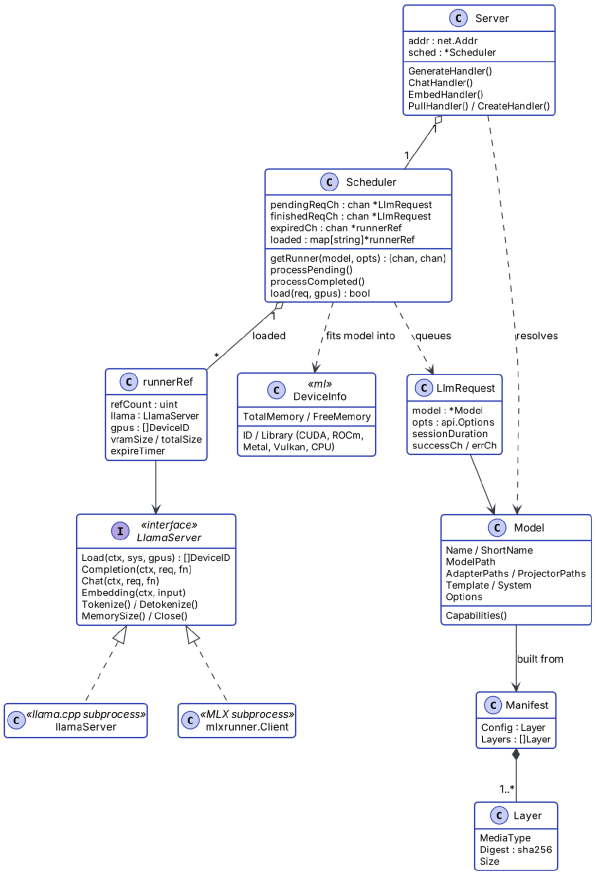


Figure 3. Logical view: key classes and their relations.

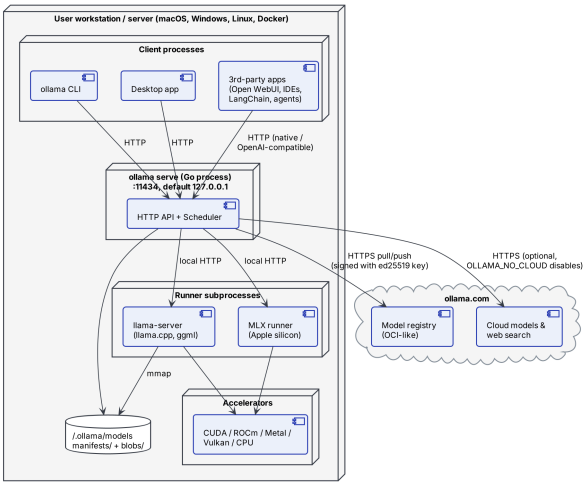


Figure 4. Deployment view.

Process view

Figure 5 follows a single chat request. The interesting part is concurrency. HTTP handlers run on many goroutines, but every load and unload decision goes through the scheduler's channels one at a time. This means no locks are needed around the GPU memory accounting, but the downside is that all model placement has to pass through this one point.

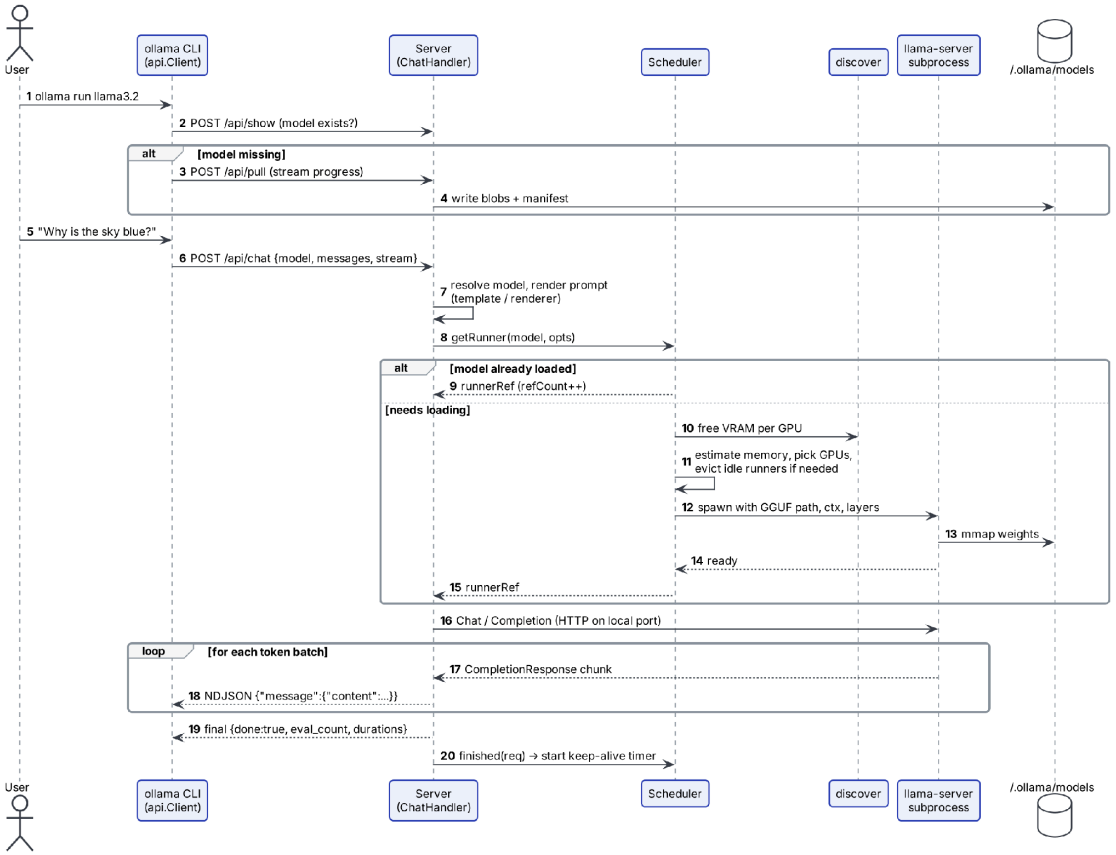


Figure 5. Process view: `ollama run` *with a cold start, then token streaming.*

2.6 Rationale behind the architecture

Based on the code and the views above, we think the main decisions were made for the following reasons.

- A client–server design even on one machine. Keeping the model behind a long-running local server means many tools (the CLI, the desktop app, editors, agents) can share one loaded copy of the weights. If Ollama were a library, every process would load the model separately.
- Go for the control plane and C/C++ for the data plane. Go makes cross-compiling easy, has goroutines and produces a small static binary. The heavy maths stays in llama.cpp and MLX, which is where the GPU expertise is.
- Runners outside the main process. Running the engine as a subprocess adds a little latency (local HTTP), but in return it gives fault isolation, lets llama.cpp be upgraded on its own and made it possible to add MLX without changing the scheduler.
- Model distribution similar to Docker. Developers already know the idea of pull, tag, push and a build file, so there is less to learn. Content addressing also gives deduplication and integrity checks for free.
- Centralised scheduling. GPU memory is the scarcest resource, so having one owner for all placement decisions keeps them consistent and easy to test. The scheduler's dependencies are passed in as functions such as `loadFn` and `getGpuFn`, so tests can fake the hardware.
- Compatibility through adapters instead of a rewrite. The native API stays simple, and the OpenAI and Anthropic formats are translated at the edge.

2.7 Architectural patterns

Pattern	Where it appears
Client-server	The CLI, the desktop app and third-party apps are all clients of `ollama serve` over HTTP.
Layered architecture	HTTP layer, then model handling, then scheduling, then the runner, then the backend library. Each layer only calls the one below it.
Master-worker	The server process supervises one worker subprocess for each loaded model.
Producer–consumer / message queue	Handlers put `LlmRequest`s on channels and the scheduler goroutine consumes them.
Strategy	`LlamaServer`, with a llama.cpp implementation and an MLX implementation chosen per model.
Adapter	The OpenAI- and Anthropic-compatible middleware wraps the native handlers.
Pipes and filters	The request path: render the template, tokenize, generate, parse thinking and tool calls, then stream.
Repository + content-addressable storage	Manifests and sha256 blobs in `~/.ollama/models`, mirrored by the remote registry.

3. Extension: an observability layer

3.1 Choice of evolution area and reasoning

In Section 2.4, observability came out as Ollama's weakest quality attribute. Right now an operator only has three tools: the server logs, `GET /api/ps` and the timing fields that come at the end of each response. None of these answer the questions operators actually have. How long do requests wait in the queue? How often are models reloaded because of eviction or out-of-memory errors? Which model and which client is using the GPU? Has p95 latency got worse after an upgrade?

There is real demand for this. The GitHub issue asking for a Prometheus `/metrics` endpoint #3144 is still open with more than a hundred comments, another issue asks for OpenTelemetry tracing #9254, and at the time of writing there are two competing pull requests with different minimal implementations #16998 and #18508. Some commenters say they keep private forks just to have metrics, and there are third-party exporters that poll the API from the outside.

We chose this area for three reasons. First, it is the most requested missing feature for shared deployments. Second, the maintainers' hesitation is about architecture (how heavy the dependencies are, where to hook in and what to expose), not about whether it would be useful. Third, it fits well with Part 4: once many users sit behind an IAM gateway, usage per user and per model becomes essential.

We set ourselves five design goals:

- G1. No cost at all when it is turned off.
- G2. Never slow down or block the inference path.
- G3. No new required dependencies.
- G4. Standard formats (Prometheus text, OTLP) so that existing tools work with it.

- G5. One instrumentation point per event, with many possible consumers.

3.2 Architectural change

We add a `telemetry` package inside the server. Existing components do not call exporters directly. Instead, they emit typed events at five points they already know about: when a request starts, the time spent waiting in the queue, model loading, token generation and when a request ends. A bounded, non-blocking event bus passes each event on to separate observers: a metrics registry, a trace builder and a usage ledger. Exporters then present this data in standard formats: Prometheus text on `GET /metrics`, OTLP spans sent to a collector, and a JSON usage summary on `GET /api/usage`, which the desktop app shows in a new "Activity" tab.

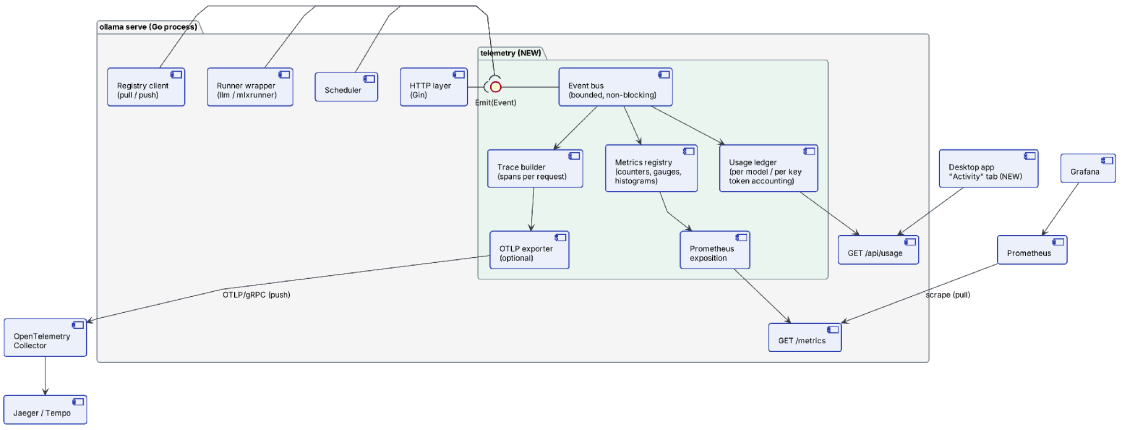


Figure 6. Component view of the extension (new parts are shaded green).

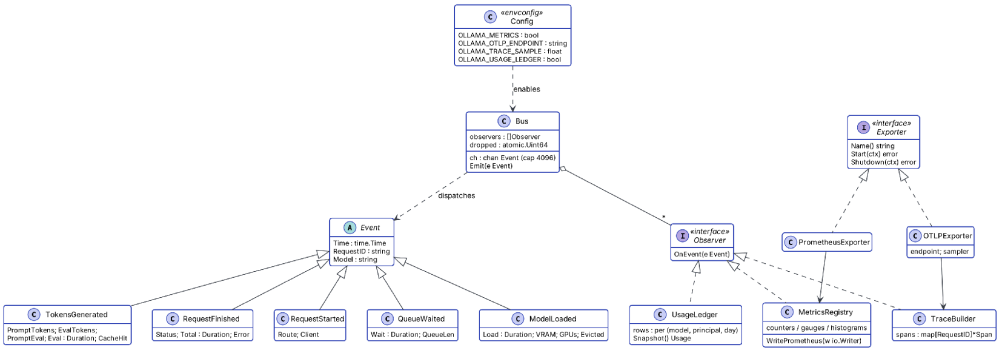


Figure 7. Logical view of the telemetry package.

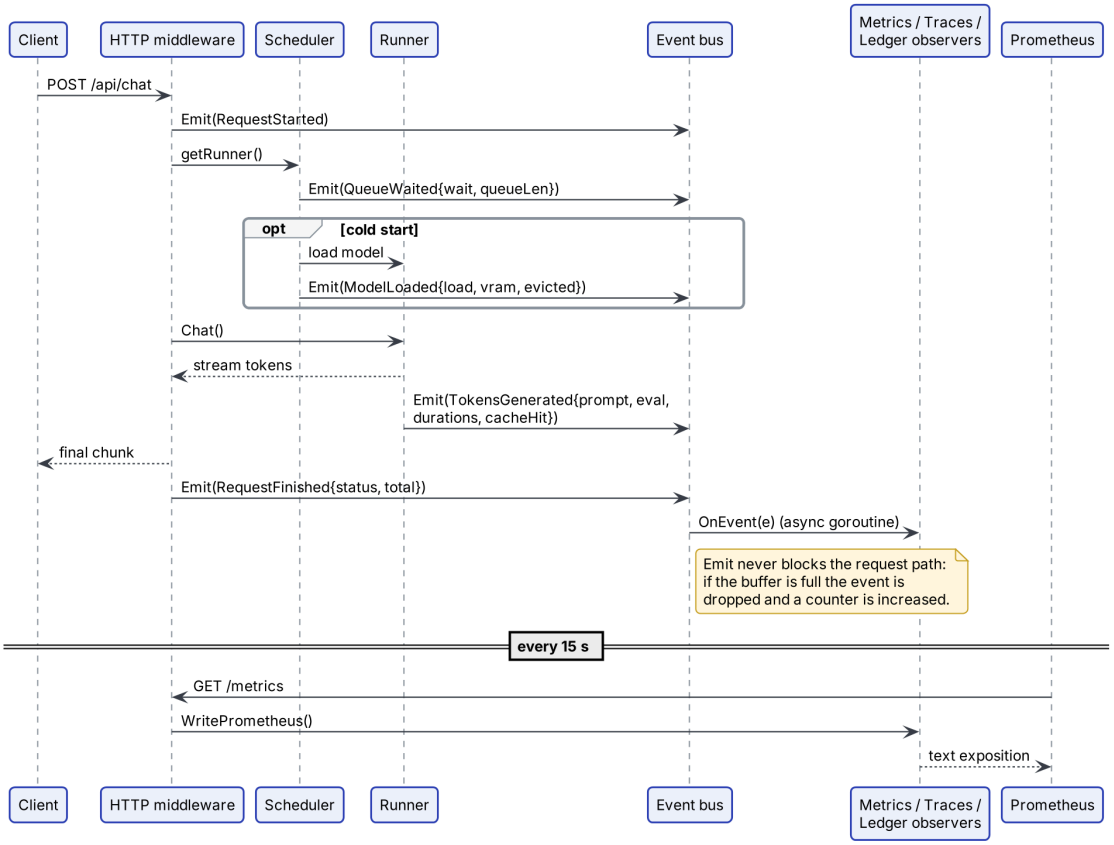


Figure 8. Process view: events emitted during one request, and a Prometheus scrape.

The metric names follow Prometheus conventions. We kept the labels low-cardinality, which means only the model and the route are used. Prompt content and user identifiers never appear on `/metrics`.

Metric	Type	What it answers
`ollama_requests_total{route,model,status}`	counter	Traffic and error rate per model.
`ollama_request_duration_seconds{route,model}`	histogram	p50, p95 and p99 latency.
`ollama_queue_wait_seconds` `ollama_queue_depth`	histogram gauge	Whether the server is saturated.
`ollama_model_load_seconds{model,reason}`	histogram	The cost of cold starts. The reason is one of first, evicted or oom_retry.
`ollama_tokens_total{model,kind}`	counter	Prompt tokens vs. generated tokens, and throughput.
`ollama_prompt_cache_hits_total{model}`	counter	How well the KV cache is being reused.
`ollama_loaded_models` `ollama_model_vram_bytes{model,gpu}`	gauge	Memory pressure on each GPU.
`ollama_telemetry_events_dropped_total`	counter	How much telemetry was dropped, so G2 is measurable.

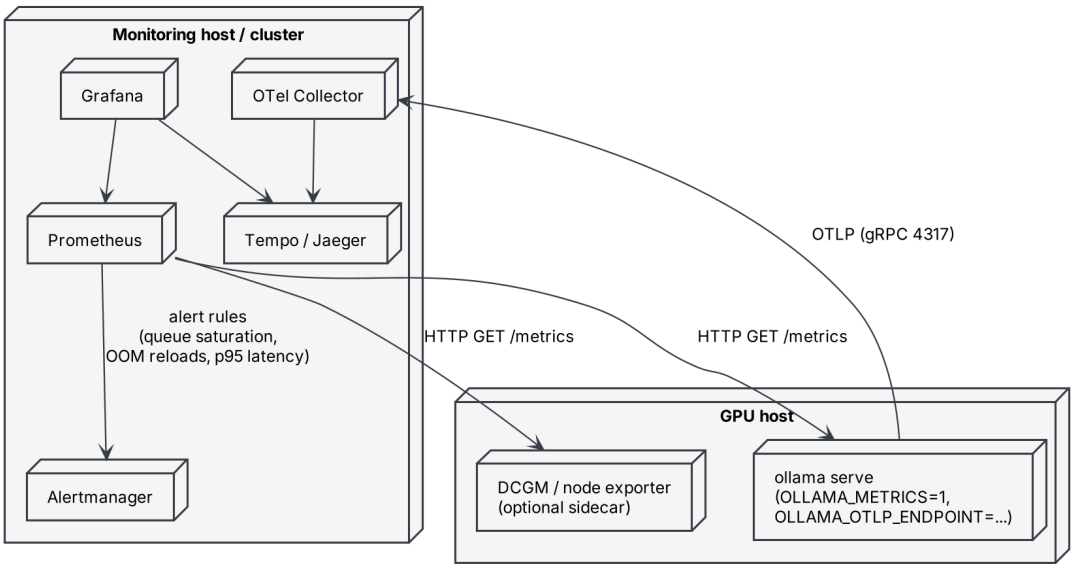


Figure 9. Physical view: a typical monitoring setup around one Ollama node.

3.3 Rationale behind the new architecture

- Events instead of direct metric calls. If the scheduler updated Prometheus counters directly, every future consumer would need another call on the same line. With events, the scheduler does not know anything about the consumers. For example, adding the usage ledger did not require any change in the scheduler.
- Emitting never blocks. `Emit` uses a `select` with a `default` branch, so if the 4096-slot buffer is full the event is dropped and counted. A slow exporter can therefore never hurt inference latency. Since dropped events are a metric themselves, the trade-off stays visible.
- Off by default, with standard formats. Setting `OLLAMA_METRICS=1` registers the observers and the endpoint. Without it, `Emit` is just a nil check. We write the Prometheus text format by hand, which is also what PR #16998 argues for, so no client library is needed. The OTLP exporter is optional and behind a build tag.
- Pull for metrics, push for traces. Pulling fits Ollama's local nature, because nothing leaves the machine unless something scrapes it. Traces are high-volume and sampled, so pushing them to a collector is the usual approach.
- The usage ledger is kept separate from the metrics. Accounting per client has high cardinality and affects privacy, so it stays out of `/metrics` and is only available through an authenticated JSON endpoint. In a gateway setup like the one in Part 4, it is keyed by the gateway's `X-Forwarded-User` header.

We also accepted some trade-offs. Under very heavy load some telemetry is lost, the histograms use a little memory for each model, and the new endpoints make the attack surface larger. Because of the last point, `/metrics` goes through the same host checks as the rest of the API.

3.4 Architectural patterns in the new architecture

Pattern	Role in the extension
Publish–subscribe / Observer	Server components publish events, and the metrics, traces and ledger subscribe to them through the `Observer` interface.

Pattern	Role in the extension
Event bus (implicit invocation)	A bounded channel separates emitters from consumers in time, and the consumers run on their own goroutine.
Strategy / plug-in	An `Exporter` interface with Prometheus and OTLP implementations, chosen by configuration.
Sidecar	Hardware-level GPU metrics stay in a separate exporter process, not inside Ollama.
Load shedding	Dropping and counting events when the bus is full protects the critical path.

4. Login page and IAM module

4.1 Requirements derived from Ollama

An IAM design only makes sense when it is based on the system it protects. By reading `server/routes.go` we got Ollama's full API surface, along with a few facts that shaped our requirements. The API has no authentication. Anyone who can reach the port can run models, but they can also pull new ones and create, copy and delete them. Responses are streamed as NDJSON or server-sent events. Exposed instances are also a known problem: Cisco's 2025 Shodan study found more than a thousand servers reachable on the internet, and many of them answered prompts without any authentication.

ID	Requirement	Where it comes from
R1	Ollama itself must not be modified, so the protection sits in front of it.	Ollama releases often, and a fork would quickly fall behind.
R2	Separate permissions for reading, inference, pulling, pushing and model management.	The route groups in `routes.go` have very different costs and risks.
R3	Model allowlists per user, using glob patterns such as `llama3.2*`.	A 70B model takes up a whole GPU, and some models have restrictive licences.
R4	People can create their own account, but only for an e-mail address they control. After that they sign in with a password and, optionally, a code from an authenticator app.	A shared lab server should not need an administrator for every new student.
R5	A new account does not get inference rights by default unless it belongs to a trusted organisation.	Inference is the scarce and costly resource (Section 2.4).
R6	Streaming responses pass through without buffering, and errors keep Ollama's `{"error": "..."}` format.	Token streaming is central to how Ollama feels, and clients already know how to show these messages.
R7	Every security decision is recorded in an audit log.	Shared lab servers need accountability.
R8	It starts with zero configuration, like Ollama (one command, SQLite, sensible defaults).	Usability is Ollama's top priority.

There are four roles, and their permissions mirror the route groups. A Viewer has `system:read` and `model:list`. A Developer also has `model:run`. A Model operator additionally has `model:pull`, `model:manage` and `audit:read`. An Administrator has everything, including `model:push` and `iam:admin`. Anyone can sign up, and the e-mail domain decides the starting role. Addresses at a configured trusted domain start as Developer. The default trusted domain is `bilkent.edu.tr`, and

subdomains such as `ug.bilkent.edu.tr` also count. Everyone else starts as a Viewer until an administrator upgrades them.

4.2 Architecture

The IAM module is a gateway that sits in front of Ollama. Ollama stays bound to 127.0.0.1, so the gateway is the only way to reach it. Inside the gateway, authorization follows the XACML split. A Policy Enforcement Point intercepts every call, asks a Policy Decision Point for a decision, and then either forwards the call or rejects it. Identities, credentials and sessions are handled by an `IAMService`. A separate `SignupService` is responsible for pending registrations and sends confirmation codes through a `Mailer` interface. Everything is written to an append-only audit log.

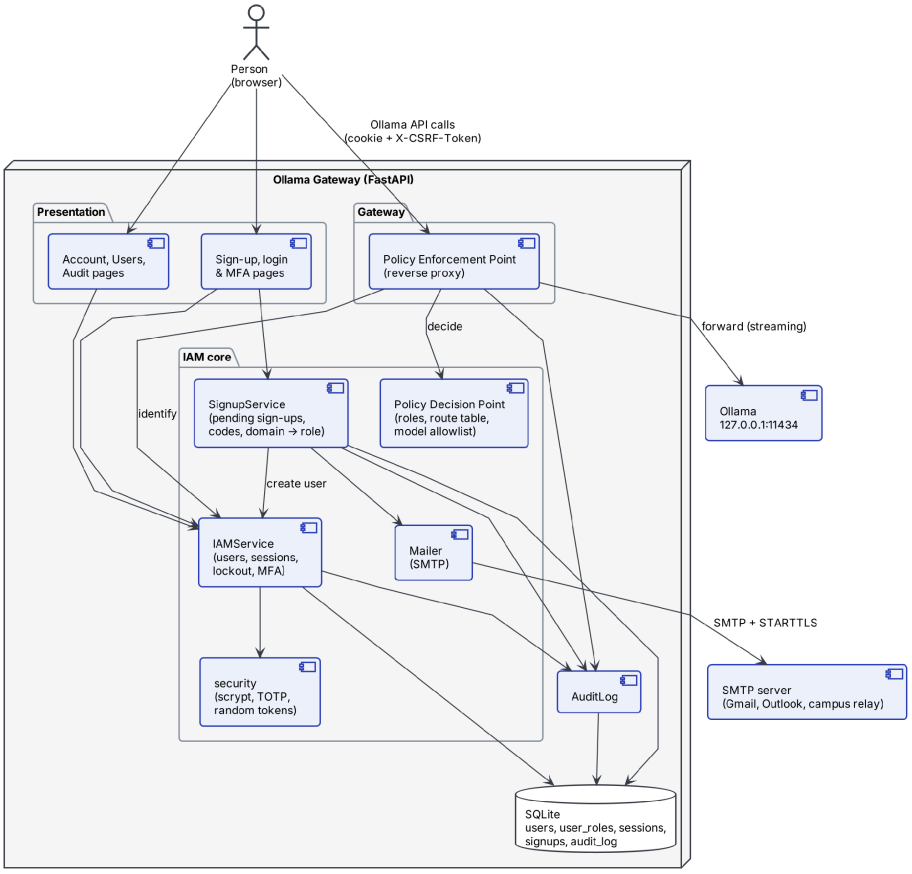


Figure 10. Component view of the gateway and IAM module.

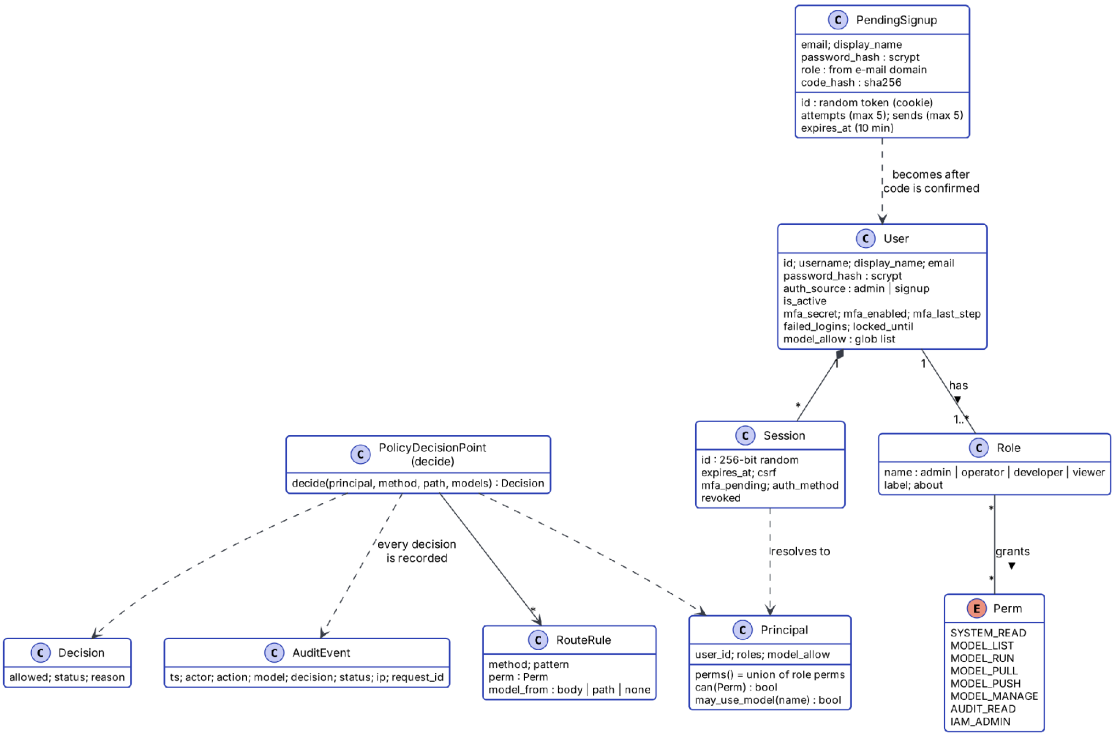


Figure 11. Logical view: the IAM domain model. A pending sign-up only becomes a user after its code is confirmed.

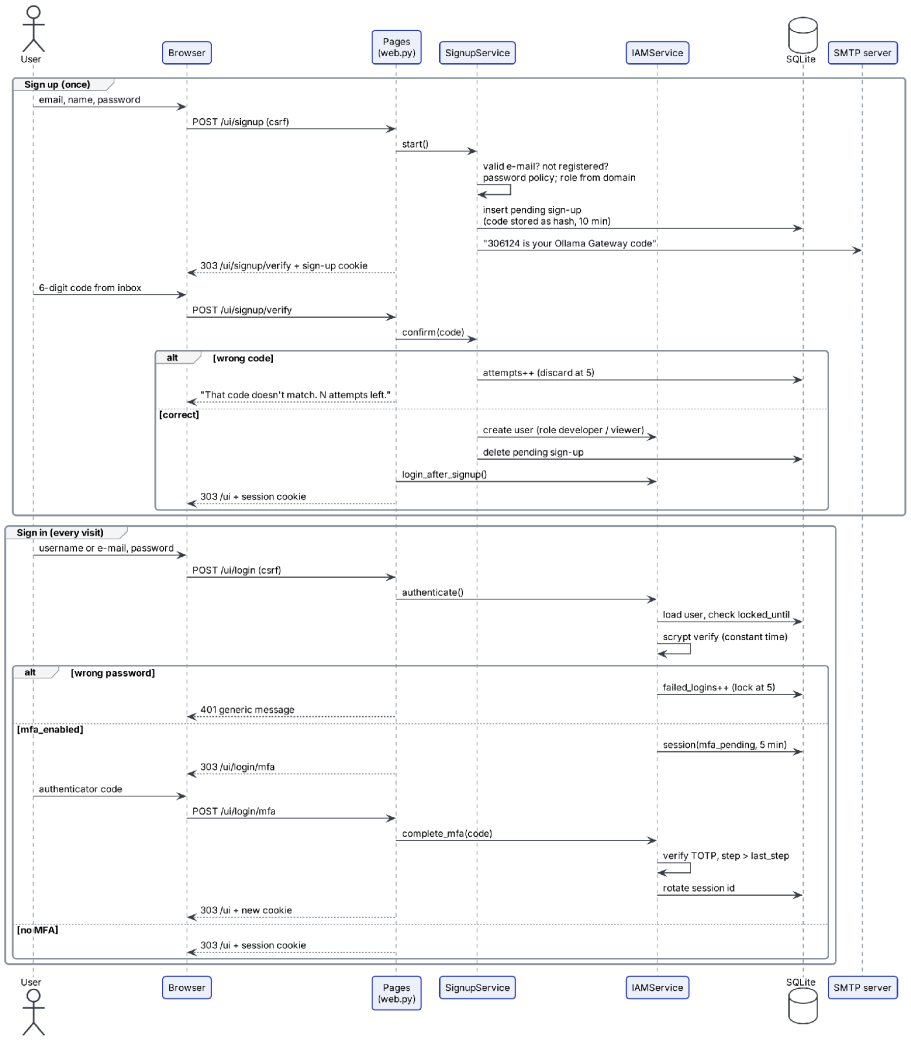


Figure 12. Process view: signing up with an e-mailed code, then signing in with an optional second factor.

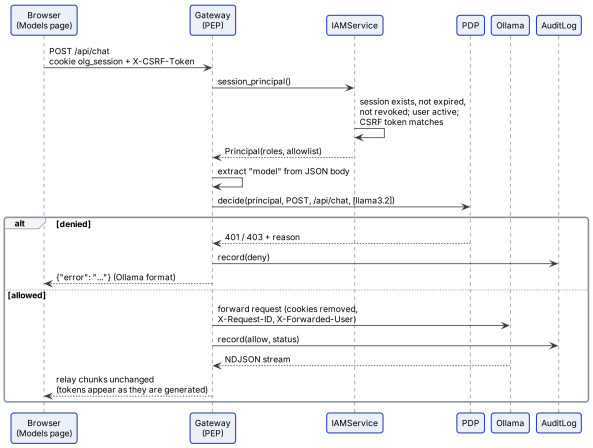


Figure 13. Process view: an API call through the enforcement point.

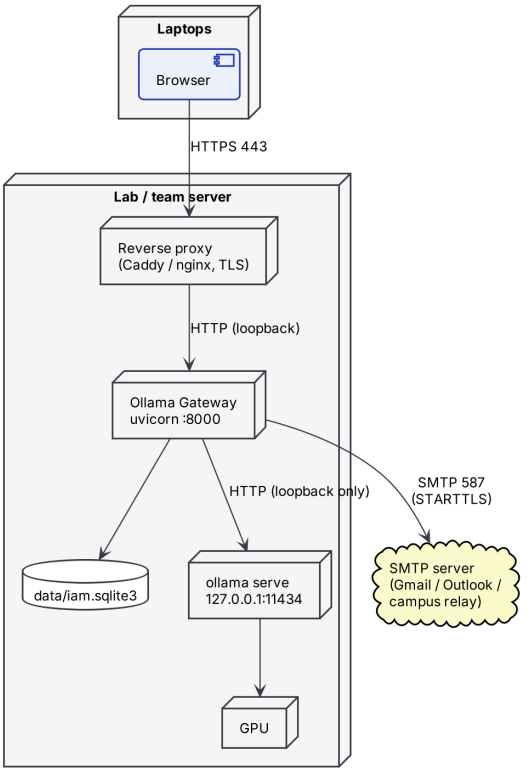


Figure 14. Physical view: the recommended deployment behind TLS, with an SMTP server for the codes.

Rationale and patterns

Decision / pattern	Why
A gateway instead of patching Ollama	This satisfies R1. It works with every Ollama release, it can be added or removed without touching the server, and it matches the Ollama FAQ, which already describes exposing the server through a proxy such as Nginx.
PEP / PDP separation	All of the access logic is one pure function, `decide()`, which is very easy to unit test. The proxy only enforces the decision.
Role-based access control + model allowlist	Roles cover the route groups, and the allowlist is an attribute check on the requested model, so it is a small RBAC and ABAC hybrid.
Route table that denies by default	Every Ollama route is listed with its permission. Anything new or unknown returns 404 until we classify it.
Layered internals	Presentation, then application services, then the repository. The proxy and the pages never touch SQL.
Pipes and filters request pipeline	Identify, extract the model, decide, forward, filter the listing, audit.
Proof of e-mail before an account exists	Pending sign-ups are kept in their own table, and nothing is added to the users table until the code is confirmed. This way, typos and fake addresses never create accounts.
Dependency inversion for e-mail	`SignupService` depends on a `Mailer` interface, not on SMTP directly. Tests pass in a fake mailer, and any provider (Gmail, Outlook, a campus relay) is just a matter of configuration.
Least privilege by default	Untrusted domains start as Viewer (R5). Inference rights for outsiders are given by an administrator, not by the sign-up form.

Decision / pattern	Why
Server-side sessions	Unlike stateless JWT sessions, they can be revoked straight away (when a user is disabled or logs out).

4.3 The login page

Ollama's own sign-in, in the desktop app, only links a local install to an ollama.com account. There is no page that protects a server. Our page is made for someone who is about to use a specific inference server, so next to the form it shows the real state of that server. It draws a route with three nodes, and the links between them turn green as the health check succeeds, along with the measured round-trip time to Ollama. If Ollama is down, the second link turns red before the user even signs in. This makes the most common support question something the user can see for themselves. The health endpoint does not reveal the version or any model names to anonymous visitors.

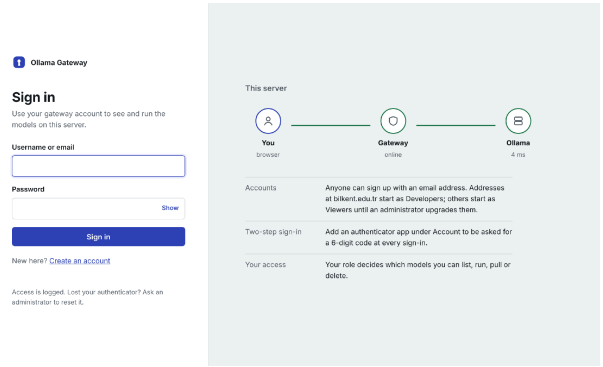


Figure 15. Sign-in page (desktop, light theme).

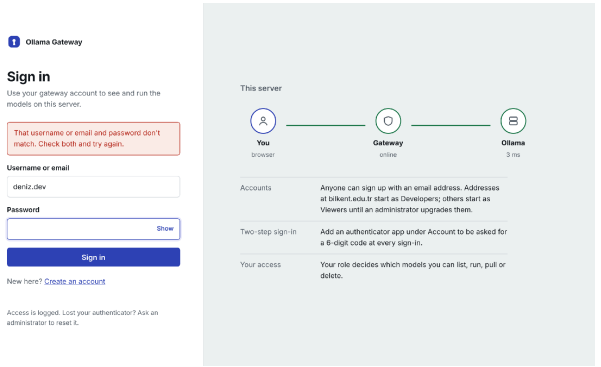


Figure 16. A failed attempt: one generic message, and the username is kept.

We also paid attention to usability. Users can sign in with either their username or their e-mail. There is a button to show or hide the password and a warning when Caps Lock is on. The submit button disables itself to stop double submissions. Error messages tell the user what to do next, and the same message is shown whether the account or the password is wrong. After five failed attempts the account is locked for 15 minutes, and the message says so. The second-factor screen has six boxes for the digits, which accept a pasted code and submit automatically.

Creating an account

Create an account asks for an e-mail, a name and a password. While the e-mail is being typed, a hint tells the person which role they will start with. Submitting the form stores a pending sign-up and e-mails a 6-digit code. The account is only created after the code is entered on the next page, and then the person is signed in straight away. Codes expire after 10 minutes, five wrong codes cancel the attempt, and "send a new code" can be used at most once a minute. If the server has no SMTP settings, the page says that sign-up is turned off, and an administrator can still add users manually.

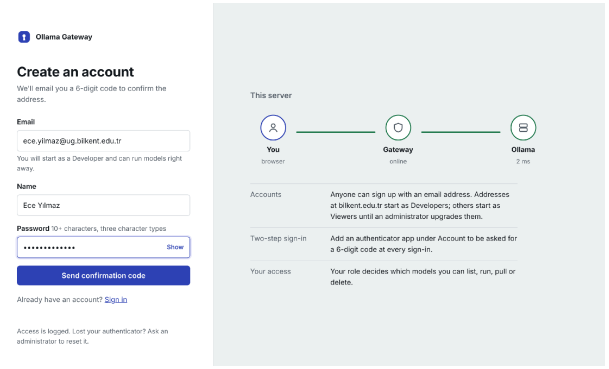


Figure 17. Sign-up form. The hint shows the starting role for a Bilkent address.

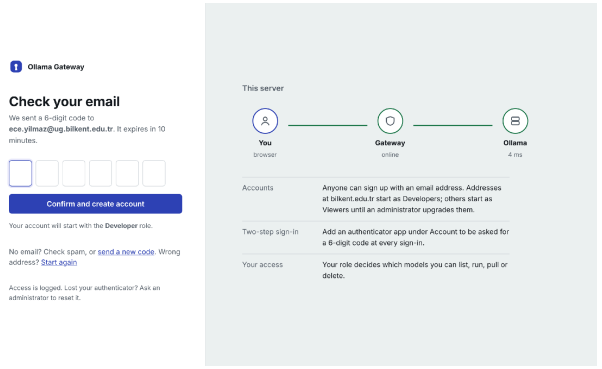


Figure 18. Code entry, with links to resend the code or start again.

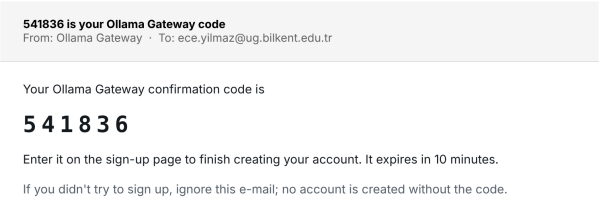


Figure 19. The confirmation e-mail as it was received over SMTP.

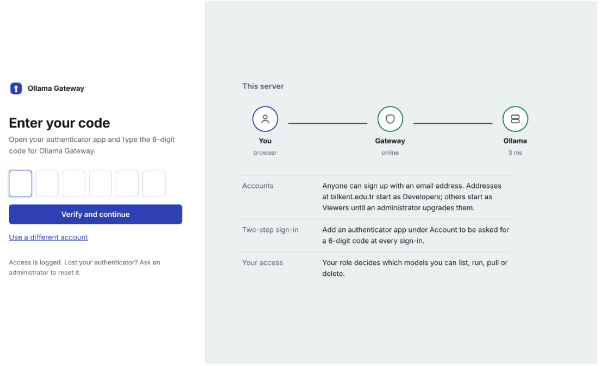


Figure 20. Optional second factor (TOTP) at sign-in.

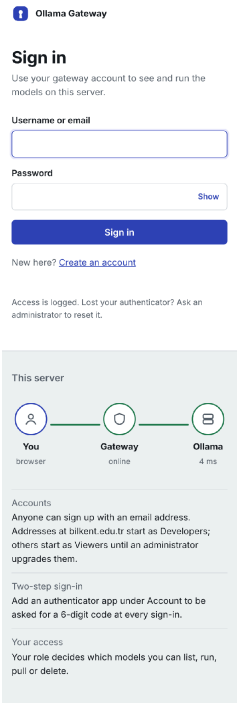


Figure 21. Phone layout.

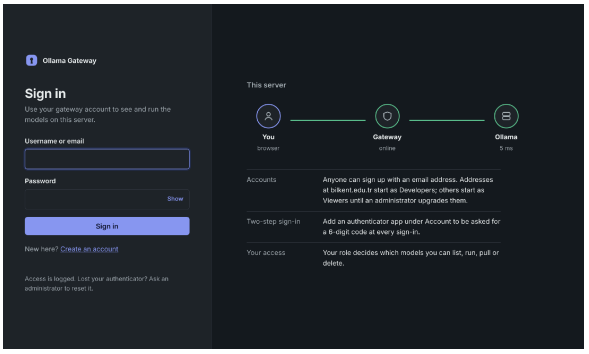


Figure 22. The dark theme follows the operating system setting.

After sign-in

The Models page only lists the models that the user is allowed to use and shows their effective permissions. It also has a small prompt box that calls Ollama through the gateway, so if the user picks a model outside their allowlist, the 403 error is shown right there (Figure 23). A Viewer can see the list but does not get a prompt box. On the Account page users can enrol an authenticator app with a QR code and change their password. Administrators can manage users, roles and model access, and read the audit log.

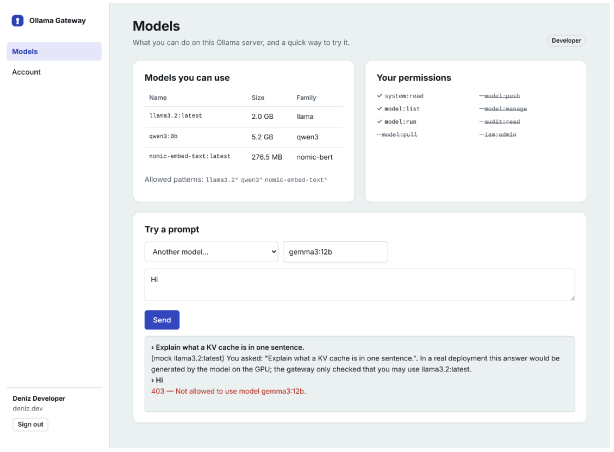


Figure 23. Developer view: allowed models, permissions, and a denied request to `gemma3:12b`.

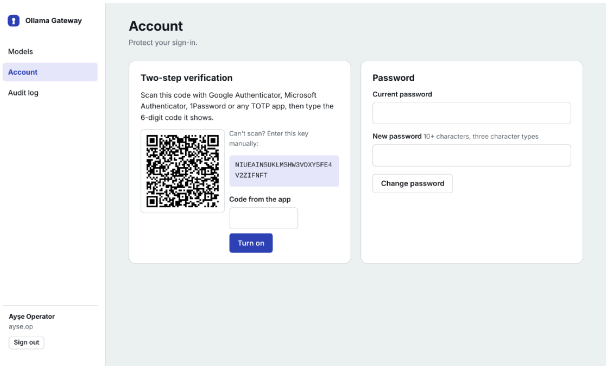


Figure 24. Enrolling an authenticator app.

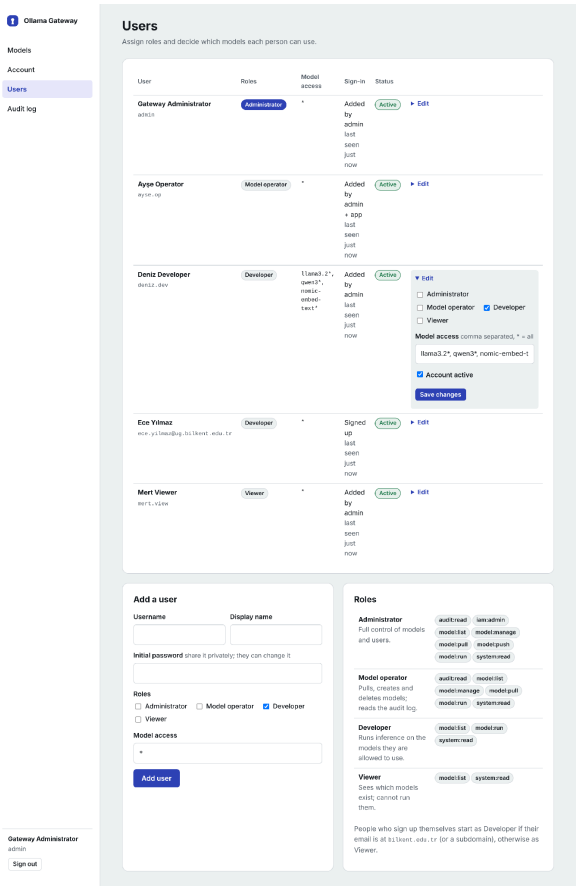


Figure 25. Administrator view: users (including the one who signed up on their own), roles and model access.

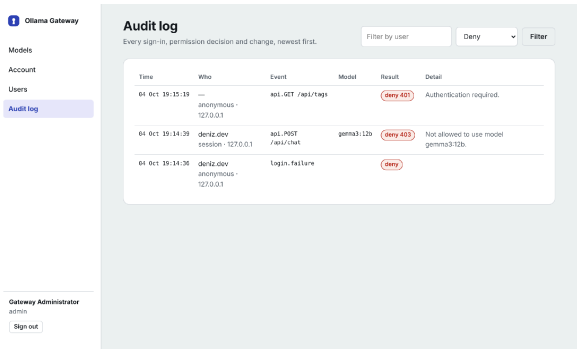


Figure 26. Audit log filtered to denials: a model outside a developer's allowlist (403), an anonymous API call (401) and a failed sign-in.

4.4 Implementation

We wrote the gateway in Python 3 with FastAPI. It is about 2,000 lines including the tests. For cryptography we only used the standard library so that every algorithm is visible: scrypt for password hashing (N = 2^{14}, r = 8, p = 1, with a salt per user), TOTP following RFC 6238 written from scratch, SHA-256 for the confirmation codes, and `smtplib` with STARTTLS for sending e-mail. Everything is stored in a single SQLite file with five tables (`users`, `user_roles`, `sessions`, `signups`, `audit_log`). The modules match Figure 10 one to one: `policy.py` is the PDP, `proxy.py` is the PEP, `iam.py` is the IAMService, `signup.py` is the SignupService, `mailer.py` is the Mailer and `web.py` is the presentation layer.

The route table is the most important part of the design, and it follows Ollama's router line by line:

```
ROUTE_RULES = [
    _r("GET",    "/api/tags",            Perm.MODEL_LIST, "none"),
    _r("POST",   "/api/chat",            Perm.MODEL_RUN),   # from body
    _r("POST",   "/v1/chat/completions", Perm.MODEL_RUN),
    _r("POST",   "/api/pull",            Perm.MODEL_PULL),
    _r("DELETE", "/api/delete",          Perm.MODEL_MANAGE),
    _r("GET",    "/v1/models/:model",    Perm.MODEL_LIST, "path"),  # from URL
    ...   # 25 rules in total
]

def decide(principal, method, path, models) -> Decision:
    rule = match_route(method, path)
    if rule is None:
        return Decision(False, 404, "Route is not exposed through the gateway.")
    if principal is None:
        return Decision(False, 401, "Authentication required.")
    if not principal.can(rule.perm):
        return Decision(False, 403, f"Missing permission {rule.perm.value}.")
    blocked = [m for m in models if not principal.may_use_model(m)]
    if blocked:
        return Decision(False, 403, f"Not allowed to use model {blocked[0]}.")
    return Decision(True, 200, "ok")
```

Choosing the starting role for a sign-up is one small rule. If the domain of the address is equal to a trusted domain, or ends with "." plus that domain, the user becomes a Developer. Otherwise they become a Viewer. Model names are normalised before matching, so nobody can get around an allowlist by writing a model name differently. The listing endpoints are filtered so users do not even see the models they are not allowed to use.

The proxy forwards requests with `httpx` in streaming mode and passes the raw chunks on, so NDJSON and SSE streams reach the client token by token. It removes the session cookie and adds `X-Request-ID` and `X-Forwarded-User` headers, which are useful for Ollama's logs and for the usage ledger we proposed in Part 3.

When the browser calls Ollama's API (for example from the prompt box on the Models page), it sends the session cookie together with an `X-CSRF-Token` header. This stops another website from making a signed-in user's browser call Ollama without them knowing. Here are two examples of rejected calls:

```
POST /api/chat  {"model":"gemma3:12b", ...}
  as deniz.dev (allowlist llama3.2*, qwen3*, nomic-embed-text*)
  -> 403 {"error": "Not allowed to use model gemma3:12b."}

POST /api/chat  {"model":"qwen3:4b", ...}
  as a Viewer
  -> 403 {"error": "Missing permission model:run."}
```

Security measures

Threat	Measure
Password guessing	The account is locked for 15 minutes after 5 failures, passwords are hashed with scrypt, and unknown users take the same time to answer.
Stolen password	Optional TOTP. A code that has been used once cannot be replayed.
Signing up with someone else's address	No account is created until the e-mailed code is entered, and the code only goes to that inbox.
Guessing a sign-up code	Codes are random, only stored as a hash and expire in 10 minutes. Five wrong tries cancel the sign-up, and resends are limited to one per minute and five in total.
Session fixation / hijacking	Random 256-bit session IDs, HttpOnly and SameSite cookies, and the ID is changed after MFA.
CSRF	There is a token in every form (with separate tokens for the login and sign-up forms), and API calls from the browser must send `X-CSRF-Token`.
Clickjacking, XSS	A Content Security Policy, `frame-ancestors 'none'` and templates that escape output automatically.

4.5 Verification and known limits

We wrote 19 automated pytest tests. They run the gateway against a stand-in for Ollama that runs in the same process and a fake mailer. The tests check the RFC 6238 test vector and that replayed codes are rejected, the policy matrix, a successful login and the generic failure message, CSRF on the login form, lockout, the full MFA flow including session rotation, unauthenticated API access, a Viewer being blocked from running models, a Developer's allowlist and filtered model list, the OpenAI-compatible route, a disabled user losing access immediately, the CSRF header for browser calls, auditing of denials, and sign-up. For sign-up we check that a Bilkent subdomain becomes a Developer, that another domain becomes a Viewer and can then sign in with its e-mail, that wrong codes are limited, that duplicates and weak passwords are rejected and that resends respect the cooldown. All 19 tests pass.

We also ran the whole sign-up flow end to end against a real SMTP server (Figure 19). The screenshots in this report were taken from that running system using a scripted browser.

The gateway still has some limits, which we list below together with how they could be fixed.

Limit	Path forward
Only browser access is covered, so scripts and SDKs cannot use the gateway.	Add scoped API keys that can be revoked, checked by the same PDP.
A single process with SQLite.	Move sessions to Redis and the database to PostgreSQL. The gateway would then be stateless and could run as several replicas.
Sign-up tells a visitor when an e-mail is already registered.	Always show "check your e-mail", and send the existing owner a "you already have an account" message instead.
The code is sent while the visitor waits, so a slow mail server makes the page slow.	Hand the messages to a background queue with retries.
Inline scripts need `'unsafe-inline'` in the CSP.	Move the scripts to static files and use nonces.

5. Conclusion

Ollama's architecture is a clear example of designing for one quality above everything else, which in this case is making local inference effortless. A Go control plane with a single scheduler, engines that run outside the main process behind one interface, model distribution modelled on Docker and adapters for other APIs at the edge all work together to achieve this. However, the same choices leave shared deployments without much support.

Our two proposals approach this gap from different sides while keeping Ollama's priorities the same. The observability layer is an opt-in, event-based addition that cannot slow down the inference path. The IAM gateway adds authentication, fine-grained authorization, auditing and a better sign-in experience without changing a single line of Ollama. The two also support each other, because the per-user identity from the gateway is what makes the usage ledger in the extension meaningful.

References

[1] Ollama source code, github.com/ollama/ollama, commit 42e911b 2 October 2026: `server/routes.go`, `server/sched.go`, `llm/server.go`, `llm/llama_server.go`, `mlxrunner/client.go`, `manifest/`, `envconfig/config.go`, `docs/faq.mdx`. MIT licence.

[2] Ollama issue #3144 "add /metrics endpoint", issue #9254 "Add support for opentelemetry tracing", and pull requests #16998 and #18508. github.com/ollama/ollama.

[3] Cisco Talos / Cisco Security Blog, "Detecting Exposed LLM Servers: A Shodan Case Study on Ollama", September 2025.

[4] Cybernews, report on CVE-2026-7482, May 2026.

[5] P. Kruchten, "The 4+1 View Model of Architecture", IEEE Software 12(6), 1995.

[6] L. Bass, P. Clements and R. Kazman, Software Architecture in Practice, 4th ed., Addison-Wesley, 2021.

[7] IETF RFC 6238, RFC 5321 and RFC 3207; OASIS XACML 3.0.

[8] Prometheus exposition format documentation; OpenTelemetry specification.